SC4242 Compiler Techniques

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Compiler Overview and Phases

> *"A compiler translates, optimises, and trusts no input."* — From characters on a screen to bytes a CPU executes, a compiler bridges two worlds by repeated, well-defined transformations.

From zero: we build here, no compiler background needed. We assume only that a program is text and a computer executes machine code. Every notion—source language, target language, phase, pass, intermediate representation—is defined from scratch with pictures before formalism. No SC2203 or SC1007 is assumed without recap.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish compiler, interpreter, and hybrid (JIT) execution models;

(L2) name and place the six core phases: lexing, parsing, semantic analysis, IR lowering, optimisation, code generation;

(L3) explain front-end vs. middle-end vs. back-end separation and why IR matters;

(L4) trace a tiny program `a = b + c*2` through tokens, tree, IR, and assembly;

(L5) reason about pass ordering, diagnostics, and what "correct compilation" means.

1.1 Why Compilers? Source, Target, and Trust

Intuition: you write in a human-friendly language (Python, C, Java); the hardware understands only numbers—opcodes, registers, addresses. A compiler is a total function compile : Source $\to$ Target that must preserve meaning while fitting hardware constraints. If it mistranslates, the bug is invisible at source level.

A *source language* is a set of legal texts defined by a grammar; a *target language* is the instruction set (e.g., ARM, x86, JVM bytecode). The compiler's contract is *semantics preservation*: for every source program P,

the observable behaviour of compile(P) equals that of P (modulo undefined behaviour).

We build this contract stepwise. Rather than one giant translation, we pipeline small, testable steps—phases—each with a precise input/output data structure.

Definition 1.1 (Phase vs. pass)**.** A *phase* is a logical task (e.g., parsing). A *pass* is a traversal over an intermediate representation (IR). One phase may be one or many passes; one pass may serve multiple phases for efficiency.

Example 1.1 (Interpreter vs. compiler vs. hybrid)**.** CPython interprets: loop reads source, executes. GCC compiles ahead-of-time (AOT): source → object file once. HotSpot JVM is hybrid: interprets first, then JIT-compiles hot methods (Ch. 8). The trade-off is start-up latency vs. peak throughput.

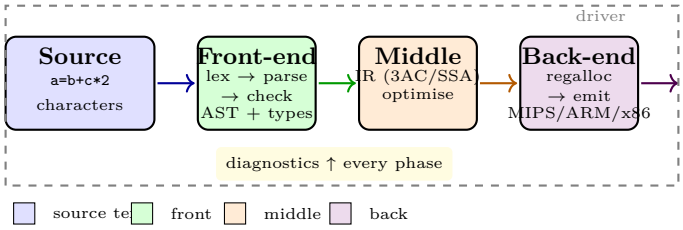


Figure 1.1: The compiler pipeline: characters → tokens → AST → IR → optimised IR → target. The driver orchestrates passes and surfaces diagnostics.

1.2 The Six Phases in Detail

We fix a tiny running language: expressions with +, ∗, assignment, integers, and variables.

Lexical analysis (Ch. 2) groups characters into *tokens*: `a`, `=`, `b`, `+`, `c`, `*`, `2`. Whitespace and comments are discarded. Output: token stream with source locations.

Syntax analysis / parsing (Ch. 3) arranges tokens into a tree per a *context-free grammar*. Leaves are tokens; internal nodes are syntactic categories. Output: parse tree → *abstract syntax tree* (AST) that drops punctuation.

Semantic analysis (Ch. 4) checks meaning: are names declared? do types align? It maintains a *symbol table* (scopes → declarations) and annotates the AST with types. Errors here are not syntax errors but "no such variable" or "cannot add string to int".

Intermediate representation (Ch. 5) lowers the AST to a simpler, uniform IR such as *three-address code* (3AC) and *SSA*. IR is where most tooling lives: it is target-independent yet close to machine.

Optimisation (Ch. 6) rewrites IR to preserve semantics while improving time/space/energy. Analyses (dataflow) determine what is safe; transforms apply it.

Code generation (Ch. 7) maps IR to target instructions, selecting instructions, scheduling them, and allocating registers (k physical from unbounded virtuals).

Definition 1.2 (Front / middle / back)**.** *Front-end* = source-dependent (lex, parse, type-check; n front-ends for n languages). *Back-end* = target-dependent (instruction selection, regalloc; m back-ends for m targets). *Middle-end* = IR + optimisation, shared: $n+m$ work instead of $n \times m$ without IR. This is why LLVM has one IR for many languages/targets.

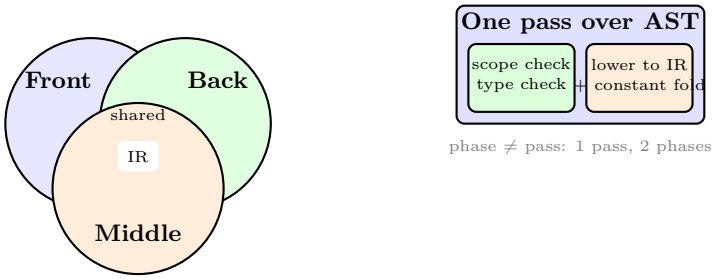


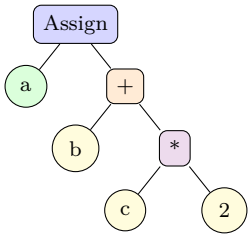
Figure 1.2: Left: front/middle/back share IR to avoid $n \times m$ compilers. Right: a pass may fuse phases (e.g., one AST walk does sema + lowering).

1.3 Running Example: `a = b + c * 2`

Intuition: the same computation takes many shapes. We trace it through each representation.

Tokens: [`ID:a`, `=`, `ID:b`, `+`, `ID:c`, `*`, `INT:2`].

AST (assignment with binary ops, respecting $*$ above $+$):



3AC (one operation per line, temporaries t_i):

$$t_1 = c * 2$$
$$t_2 = b + t_1$$
$$a = t_2$$

Optimised? No change here; constant folding would replace $c * 2$ only if c known constant. Register-allocated MIPS (assuming a, b, c in memory):

```
# MIPS-like: $t0,$t1 temporaries, $a0 holds &a etc. (illustrative)
lw $t0, b
lw $t1, c
sll $t1, $t1, 1   # c*2 == c <<1  (strength reduction, Ch.6)
add $t0, $t0, $t1
```

```
6 sw $t0, a
```

Diagnostics must point at source: if `c` undeclared, the error at line:col of `c`, not at `t1`.

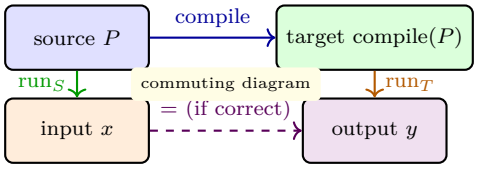


Figure 1.3: Correct compilation: running P on x at source level equals running compile(P) on x at target level (for defined behaviours). Optimisations must keep the square commuting.

1.4 Driver, Diagnostics, and Toolchain

A *compiler driver* (e.g., `clang`, `gcc`) orchestrates: preprocess → compile → assemble → link. It invokes the compiler proper (source→object) then linker (objects+libs→executable). Options select optimisation levels ($-O0$ vs. $-O2$), debug info, and warnings.

Diagnostics are not afterthoughts: keep source locations through every IR; emit "error: use of undeclared identifier 'c' at 3:9" with a snippet, not "IR tmp t1 undefined". Modern compilers store *spans* per node.

Example 1.2 (Driver in 20 lines).

```
# Highly simplified driver; real one handles temp files, parallel jobs, caching
def compile_file(src_path, opt_level=2):
    text = open(src_path).read()
    toks = lex(text, filename=src_path)        # Ch.2: LexResult with spans
    ast  = parse(toks)                          # Ch.3: may raise ParseError
    sema(ast)                                   # Ch.4: name/type errors here
    ir   = lower(ast)                           # Ch.5: AST -> 3AC/SSA
    if opt_level >= 1: ir = optimise(ir)        # Ch.6
    asm  = codegen(ir, target="aarch64")        # Ch.7
    obj  = assemble(asm)
    return link([obj], libs=["libc"])
```

What can go wrong?

Formal semantics helps: undefined behaviour (e.g., signed overflow in C) means the commuting square need not hold—the compiler may assume it never happens and optimise accordingly. Phase ordering matters too: constant propagation enables dead-code elimination, but not vice versa.

1.5 Chapter Notes

Language design and compilation co-evolve: Fortran (1957) invented the compiler; Lisp introduced trees as code; C exposed the machine; Rust makes ownership checkable at compile time. The front/middle/back split matured in LLVM (Lattner, 2004) and GCC. For theory, SC2203's regular/CF languages are the foundation;

for practice, Appel's *Modern Compiler Implementation* and Aho–Lam–Sethi–Ullman (*Dragon Book*) are the two complementary bibles—this textbook follows Appel's structure with Dragon-Book rigour.

1.6 Exercises

E1.1 **Phases vs. passes.** Give a one-pass design that fuses semantic checking and IR lowering over the AST. What coupling cost does it pay versus two separate passes? Sketch both traversals as visitor pseudocode.

E1.2 **Token trace.** Lex `sum = x1_2 + 0xFF // comment` assuming identifiers `[a-zA-Z_][a-zA-Z0-9_]*`, hex `0x[0-9a-fA-F]+`, and `//` line comments. List tokens with lexemes and argue why `x1_2` is one token not three.

E1.3 **AST vs. parse tree.** Draw full parse tree and then AST for `a = b + c * 2` under grammar $E \to E + E \mid E * E \mid$ `id`. Mark dropped nodes and explain why the AST is sufficient for later phases.

E1.4 **Front/middle/back counting.** With 5 source languages and 4 targets, how many compilers without IR vs. with one shared IR? If IR is lossy (cannot express one language's feature), what cost appears? Propose a fix.

E1.5 **Correctness with UB.** C says signed overflow is undefined. Show how `for(i=0;i<=N;i++) a[i]=0` with `N==INT_MAX` can be miscompiled if the compiler assumes `i+1 > i` always. What does the commuting diagram say?

Chapter 2

Lexical Analysis: From Regex to DFA

"Scanning is pattern matching made deterministic." — Regular expressions describe tokens; NFAs execute them speculatively; DFAs execute them deterministically at bytes-per-cycle speed.

From zero: we build here, no automata assumed beyond sets. We define alphabets, languages, and regular expressions from scratch (recapping SC2203 Ch. 2 in one page), then build Thompson NFAs, ε-closure, subset construction, and minimisation. Pictures precede formalism throughout.

Learning Outcomes

After this chapter you will be able to:

(L1) write regular expressions for tokens (keywords, identifiers, numbers, strings);

(L2) build a Thompson ε-NFA from a regex and argue its size is linear;

(L3) compute ε-closure and simulate an NFA on an input string;

(L4) convert NFA→DFA via subset construction and minimise via partition refinement;

(L5) implement maximal-munch lexing with priority and emit a token stream with spans.

2.1 Regular Expressions for Tokens

Intuition: a token is a *lexeme* matched by a pattern: "identifier looks like a letter followed by letters/digits". Regexes name those patterns precisely.

Fix alphabet Σ (e.g., ASCII 0–127). A *language* is a set $L \subseteq \Sigma^*$.

Definition 2.1 (Regex syntax and $L(\cdot)$)**.** Basis: $\emptyset$ with $L = \emptyset$, ε with $L = \{\varepsilon\}$, a ($a \in \Sigma$) with $L = \{a\}$. If R, S are regexes then $(R|S)$ is union ($L(R) \cup L(S)$), (RS) concatenation ($\{xy \mid x \in L(R), y \in L(S)\}$), (R^*) star ($\bigcup_{n \geq 0} L(R)^n$). Shorthand: $R^+ = RR^*$, $R? = R|\varepsilon$, [a-z] enumerates, $R\{m, n\}$ bounded repeat, "." is $\Sigma \setminus \{\backslash n\}$.

Precedence: $*$ highest, concat next, | lowest; so `ab|cd*` means `(ab)|(c(d*))`.

Example 2.1 (Token patterns)**.** `if` keyword: `if` (exact). Identifier: `[a-zA-Z_][a-zA-Z0-9_]*`. Integer: `0|[1-9][0-9]*`. Hex: `0x[0-9a-fA-F]+`. String: `"[^"]*"` (ignoring escapes for now). Whitespace/comments are tokens too, just discarded.

A lexer has *many* regexes $R_1, \ldots, R_k$ (one per token kind). We combine them as $R = (R_1|R_2|\ldots|R_k)$ tagged by kind and priority (keywords beat identifiers when both match `if`).

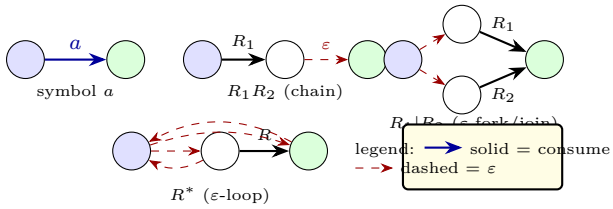


Figure 2.1: Thompson fragments: each regex maps to an ε-NFA with one entry and one exit, built inductively in $O(|R|)$ states/edges.

2.2 From Regex to ε-NFA

Intuition: an NFA may be in many states at once—"guess" which R_i matches. That nondeterminism is exactly what regex union and star mean.

Definition 2.2 (Thompson NFA)**.** For each regex node build fragment above with fresh entry/exit. ε-edges are unlabeled. Sizes add linearly: $|\text{NFA}(R)| \leq 2|R|$ states. See Fig. 2.1.

Example 2.2 (NFA for `[a-b]*abb`)**.** Thompson gives 2 states per literal plus forks for star and union. The full NFA has ≈ 14 states before trimming; it has parallel a/b branches inside the star. Any regex of length n yields $O(n)$-size NFA in linear time.

```python
# Thompson: AST node -> (entry, exit, edges)
def thompson(node, new_state):
    if node.kind == 'lit':          # 'a'
        s, t = new_state(), new_state()
        return (s, t, [(s, node.char, t)])
    if node.kind == 'concat':
        l = thompson(node.left, new_state); r = thompson(node.right, new_state)
        return (l.entry, r.exit, l.edges + r.edges + [(l.exit, None, r.entry)])
    if node.kind == 'alt':          # R|S : epsilon fork/join (Fig.1)
        s, t = new_state(), new_state()
        l = thompson(node.left, new_state); r = thompson(node.right, new_state)
        eps = [(s,None,l.entry),(s,None,r.entry),(l.exit,None,t),(r.exit,None,t)]
        return (s, t, l.edges + r.edges + eps)
    if node.kind == 'star':
        s, t = new_state(), new_state()
        c = thompson(node.child, new_state)
        eps = [(s,None,c.entry),(s,None,t),(c.exit,None,c.entry),(c.exit,None,t)]
        return (s, t, c.edges + eps)
```

2.3 Subset Construction: NFA → DFA

To run fast, we remove guessing.

Definition 2.3 (ε-closure and move)**.** Eclose(S) = all states reachable from S via ε^*. move$(S, a) = \{t \mid \exists s \in S : s \xrightarrow{a} t\}$. DFA state = subset of NFA states: $Q_D = \mathcal{P}(Q_N)$ (only reachable subsets built lazily).

Transition: $\delta_D(S, a) = \text{Eclose}(\text{move}(S, a))$. Start $q_{0D} = \text{Eclose}(\{q_{0N}\})$. Accepting if $S \cap F_N \neq \emptyset$ (priority = smallest R_i whose accept in S).

Example 2.3 (Ends-with-`01`: NFA 4 states → DFA 4 reachable of 16)**.** NFA $q_0 \xrightarrow{0,1} q_0$, $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$ accept. Subset walk: $\{q_0\} \xrightarrow{1} \{q_0\}$, $\xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\}$ accept, etc. Fig. 2.2 shows lattice climb.

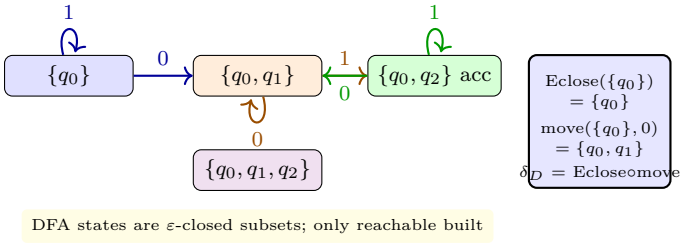


Figure 2.2: Subset construction for `.*01`: DFA subsets, transitions $\delta_D(S, a)$, and ε-closure (here trivial; with ε it expands subsets). Hopcroft then merges equivalent DFA states.

Minimisation (Hopcroft). Start partition $\{F, Q \setminus F\}$; repeatedly split blocks where a-successors land in different blocks. Runs $O(|Q| \log |Q|)$ and yields the unique minimal DFA. Lexer tables shrink dramatically.

2.4 Lexing: Maximal Munch, Priority, Spans

With DFA in hand, lexing is a loop: from current input position p, run DFA feeding characters until stuck; last accept position gives longest prefix (maximal munch). If tie (e.g., `if` matches both keyword and identifier regex), choose smallest-index regex (priority). Emit token with span $[p, q)$, advance to q, skip whitespace/comments or emit them for tooling. On no accept, emit error at p and resynchronise (e.g., skip to next whitespace).

Example 2.4 (Lexing `if (x<=10)`)**.** DFA run: `if`→ keyword `IF` (priority over `ID`), `(`→ `LPAREN`, `x`→ `ID`, `<=`→ `LE` (longest: `<` alone is `LT` but `<=` longer), `10`→ `INT`. Token stream fed to parser; whitespace dropped but line:col kept.

```
def lex(dfa, text, kinds):
    pos, n, out = 0, len(text), []
    while pos < n:
        state, last_accept, last_pos = dfa.start, None, pos
        i = pos
        while i < n and (state := dfa.delta.get((state, text[i]))) is not None:
            i += 1
            if state in dfa.accept:           # remember last accept
                last_accept, last_pos = state, i
```

```
        if last_accept is None:                # no token
            raise LexError(f"illegal char {text[pos]!r} at {pos}")
        kind = kinds[last_accept]              # priority already in accept tag
        lexeme = text[pos:last_pos]
        if kind not in ('WS','COMMENT'):
            out.append((kind, lexeme, pos, last_pos))
        pos = last_pos                         # maximal munch advance
    return out
```

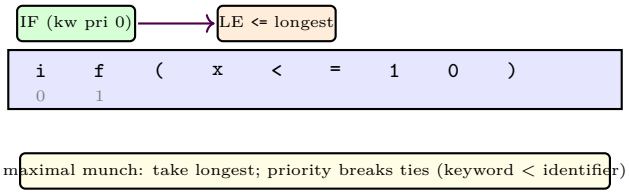


Figure 2.3: Lexing decisions: longest prefix wins; on equal length, earliest regex (keywords) wins. Spans $[l, r)$ kept per token for diagnostics and incremental re-lexing.

2.5 Chapter Notes

Thompson (1968) introduced regex→NFA for QED editor; subset construction is Rabin–Scott (1959); Hopcroft (1971) $O(n \log n)$ minimisation is optimal. Flex and RE2 still use Thompson + DFA with on-demand subset. Unicode, UTF-8 decoding, and longest-match with look-ahead (`/` trailing context) extend the core; RE2 and Rust's regex crate document production concerns.

2.6 Exercises

E2.1 **Regex to ε-NFA.** Build Thompson NFA for $(a|b)^*abb$; count states/ε-edges. Show ε-closure of start and trace `ababb`.

E2.2 **Subset by hand.** Given NFA $q_0 \xrightarrow{\varepsilon} q_1$, $q_0 \xrightarrow{a} q_0$, $q_1 \xrightarrow{b} q_2$ accept, compute reachable DFA subsets, table δ_D, and minimise. Which 2 states merge?

E2.3 **Maximal munch counterexample.** Regexes: `IF if` pri 0, `ID [a-z]+` pri 1, `WS [ ]+` pri 2. Lex `iff iffy if`. List tokens and state where priority vs. length matters.

E2.4 **Keyword vs. identifier DFA.** Construct combined NFA for {`if`,`int`,`[a-zA-Z_]\w*`} with tagged accepts, subset-construct, show why `if` is not `ID` (tag = smallest index). What NFA/DFA size?

E2.5 **Error recovery.** Input `"unterminated` (no closing quote) stalls DFA with no accept. Design two policies: (a) error token to next quote, (b) single-token resync to next whitespace; trade-offs for diagnostics and parser recovery.

Chapter 3

Parsing: Grammars, LL and LR, and ASTs

"Syntax is shape: parsing recovers the tree hidden in the token line." — From a flat stream to a hierarchical tree, respecting precedence and nesting.

From zero: we build here, no grammar assumed. We define alphabets, strings, context-free grammars, derivations, ambiguity, and trees from first principles (recapping SC2203 CFGs in two pages). FIRST/FOLLOW, LL(1) and LR parsing are motivated by pictures then tables.

Learning Outcomes

After this chapter you will be able to:

(L1) define CFG, derivation, sentential form, parse tree, ambiguity, and the language $L(G)$;

(L2) compute nullable, FIRST, and FOLLOW sets and construct an LL(1) table;

(L3) construct LR(0) items, canonical collection, and SLR/LALR action/goto tables;

(L4) build an AST from a parse (desugaring, precedence, associativity) and explain error recovery;

(L5) choose LL vs. LR for a language feature and justify with a grammarTransform.

3.1 Context-Free Grammars

Intuition: tokens are leaves; grammar rules are "a phrase A may be $X_1 X_2 \dots X_k$". Repeated expansion grows a tree whose yield (leaves left-to-right) is the program.

Definition 3.1 (CFG)**.** $G = (N, T, P, S)$: N nonterminals, T terminals (tokens), P productions $A \to \alpha$ with $A \in N$, $\alpha \in (N \cup T)^*$, start $S \in N$. Write $A \to \alpha_1 \mid \alpha_2$. A *derivation* $S \Rightarrow^* w$ rewrites leftmost (or any) nonterminal using P; $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$. A *parse tree* has root S, internal nodes A, children spelling a production, yield w.

Example 3.1 (Expressions, naïvely ambiguous)**.** $E \to E + E \mid E * E \mid (E) \mid \texttt{id} \mid \texttt{num}$. Then $\texttt{id} + \texttt{id} * \texttt{id}$ has two parse trees (+ on top vs. $*$ on top). Ambiguity = two different meanings (precedence).

Example 3.2 (Derivation as tree construction)**.** Leftmost derivation for $\texttt{id} + \texttt{id} * \texttt{id}$ via layered grammar: $E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow \texttt{id} + T \Rightarrow \texttt{id} + T * F \Rightarrow \texttt{id} + F * F \Rightarrow \texttt{id} + \texttt{id} * F \Rightarrow \texttt{id} + \texttt{id} * \texttt{id}$. Each $\Rightarrow$ chooses a production; the tree built alongside has internal nodes E, T, F recording which choice. Yield is read off leaves. Ambiguous grammars allow two such sequences with different trees but same yield.

We fix it by layering: $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid \texttt{id} \mid \texttt{num}$. Now $*$ binds tighter, + left-associative, and the grammar is unambiguous. Cross-check: yield of tree rooted at E for $\texttt{id} + \texttt{id} * \texttt{id}$ is unique with $*$ lower (closer to leaves).

Nullable, FIRST, FOLLOW (for LL/LR). nullable(A) if $A \Rightarrow^* \varepsilon$. FIRST($\alpha$) $= \{a \in T \mid \alpha \Rightarrow^* aw\} \cup \{\varepsilon$ if $\alpha \Rightarrow^* \varepsilon\}$. FOLLOW($A$) $= \{a \mid S \Rightarrow^* \beta A a \gamma\} \cup \{\$\}$ (\$= end-of-input). Computed by least fixed-point iterating productions until stable.

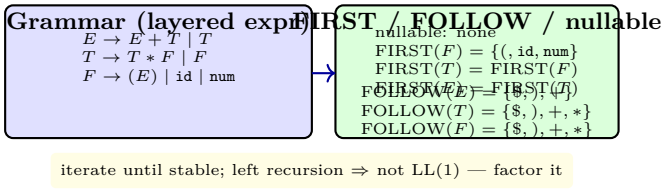


Figure 3.1: Layered expression grammar and its FIRST/FOLLOW (fixed-point). Left recursion makes it non-LL(1); factoring or LR handles it.

3.2 LL Parsing: Predictive Top-Down

Intuition: look at next token ($k = 1$) to decide which production to expand—like choosing a road by the next sign.

After eliminating left recursion and left-factoring, build table $M[A, a]$: for $A \to \alpha$, for each $a \in$ FIRST(α) set $M[A, a] = A \to \alpha$; if $\varepsilon \in$ FIRST(α), for each $a \in$ FOLLOW(A) add $A \to \alpha$. LL(1) iff no conflict (one entry per cell).

Example 3.3 (LL parse of $\texttt{id} + \texttt{id}$)**.** Stack $[E, \$]$, input $\texttt{id} + \texttt{id}\$$. $M[E, \texttt{id}] = E \to T$, push T, etc. Steps alternate expand (nonterminal on stack) and match (terminal equals lookahead, consume). Fig. 3.2 shows three snapshots.

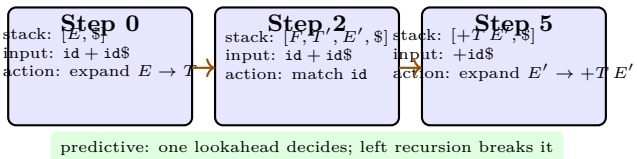


Figure 3.2: LL(1) stack simulation: lookahead ∈ FIRST determines the production; FOLLOW used when nullable.

Recursive descent. Each A is a function that inspects lookahead and calls others. Simple, debuggable, used by Clang/Go for most constructs; LR tables handle the rest.

```python
# Recursive-descent for factored expr: E -> T E';  E' -> +T E' | eps
def parse_E(toks):
    node = parse_T(toks)
    while peek(toks) == '+':
        consume(toks, '+')
        rhs = parse_T(toks)
        node = BinOp('+', node, rhs)   # left-assoc via loop, not recursion
    return node
def parse_T(toks):
    node = parse_F(toks)
    while peek(toks) == '*':
        consume(toks, '*')
        node = BinOp('*', node, parse_F(toks))
    return node
```

3.3 LR Parsing: Bottom-Up Shift-Reduce

Intuition: read tokens left-to-right, *shift* them onto a stack; when top spells a production's RHS, *reduce* it to LHS—like assembling puzzle chunks greedily but with a DFA guiding decisions.

LR(0) *item* $A \to \alpha \bullet \beta$ = "have seen α, expect β". Closure adds items for nonterminals after $\bullet$; goto on X moves $\bullet$ over X then closes. The canonical collection is a DFA over stacks; action/goto tables drive Fig. 3.3.

SLR uses FOLLOW to allow reduce only when lookahead $\in$ FOLLOW(LHS); LALR merges states with same core to shrink tables (what Yacc/Bison emit).

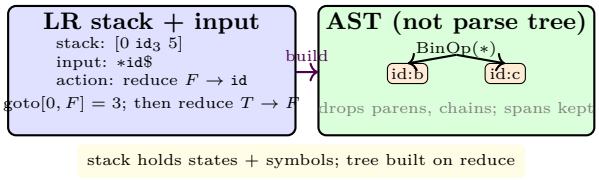


Figure 3.3: LR shift-reduce (states on stack) and the resulting AST. The AST discards concrete punctuation but retains precedence, associativity, and source spans.

```python
def lr_parse(tokens, action, goto):
    stack = [0]                 # states
    syms = []                   # symbols + AST nodes
    i = 0
    while True:
        st = stack[-1]; tok = tokens[i].kind if i < len(tokens) else '$'
        act = action[st, tok]
        if act[0] == 'shift': stack.append(act[1]); syms.append(tokens[i]); i+=1
        elif act[0] == 'reduce':
            A, n, builder = act[1], act[2], act[3]   # A -> ... (n symbols)
            args = syms[-n:] if n else []; del syms[-n:]; del stack[-n:]
            syms.append(builder(*args))                # make AST node
            stack.append(goto[stack[-1], A])
        elif act[0] == 'accept': return syms[0]
```

```
        else: raise ParseError(f"unexpected {tok} at {tokens[i].lexeme!r}")
```

AST design. Nodes: `BinOp(op,left,right)`, `Assign(name,expr)`, `If(cond,then,else)`, `Let(decls,body)`. Store span $[l, r)$ and type slot; desugar `a+=b` $\to$ `a=a+b` during parse; keep original span for diagnostics. For declarations, `Let(x, e1, e2)` scopes `x` in `e2` only (Ch. 4). Pretty-printing the AST should recover source up to whitespace; round-trip parse(print(t)) $= t$ is a property test.

Derivations vs. trees. A *sentential form* is any string derivable from S (mix of terminals and nonterminals). *Leftmost* derivation always expands the leftmost nonterminal; *rightmost* is reverse. LR traces a rightmost derivation in reverse (reductions correspond to derivation steps). That duality explains why LR handles left recursion naturally (it reduces after seeing the whole left-extended handle).

Precedence climbing (alternative). Instead of layered E, T, F, many compilers parse expressions with a precedence table:

Prec	Assoc	Ops
10	left	$*$ /
9	left	$+$ $-$
8	non	$<$ $>$ `==`
2	right	$=$

A Pratt parser loops: parse prefix, then while next op's precedence $>$ current binding power, consume op and parse next atom with higher power. Equivalent power to LR but hand-written.

Error recovery. Panic mode: on error, pop until a state with goto on a synchronising nonterminal (e.g., `Stmt`), skip input until `;/}/$`, resume. Production compilers recover many errors per file, not one-and-done. Recovery must *not* invent trees that mislead later phases; often insert an `Error` node with span covering the skipped region so sema can skip it but diagnostics point correctly.

3.4 Chapter Notes

LL and LR were classified by Knuth (1965) and Lewis-Stearns. LL(k) is simple but not all deterministic languages; LR(1)/LALR(1) cover all deterministic CFGs. Tools: ANTLR (LL(*)), Bison (LALR), Tree-sitter (GLR for error tolerance). For compilers, favour LR/LALR for expressions and declarations, recursive descent with precedence climbing for readability. The classic "dangling else" ($S \to$ `if` E `then` S | `if` E `then` S `else` S) is ambiguous; LR parsers resolve it by preferring shift over reduce (else binds to nearest if).

3.5 Exercises

E3.1 **FIRST/FOLLOW by fixpoint.** For $S \to AB$, $A \to aA \mid \varepsilon$, $B \to bB \mid \varepsilon$, compute nullable, FIRST, FOLLOW iteratively. Show tables and build LL(1) M; is it LL(1)?

E3.2 **Left-recursion elimination.** Transform $E \to E + T \mid T$, $T \to T * F \mid F$ to LL(1)-friendly $E \to TE'$, $E' \to +TE' \mid \varepsilon$, similarly for T. Recompute FIRST/FOLLOW and M; trace `id` + `id` ∗ `id`.

E3.3 **LR(0) collection.** Build LR(0) items for $S' \to S$, $S \to (S) \mid$ `id`. Draw DFA with closure/goto; list reduce/reduce or shift/reduce conflicts? Then compute SLR table.

E3.4 **Ambiguity proof.** Prove $E \to E + E \mid E * E \mid$ `id` ambiguous by exhibiting two distinct leftmost derivations and parse trees for `id` + `id` ∗ `id`. Then show layered grammar is unambiguous for length-3 yields.

E3.5 **AST + error recovery.** Design AST nodes for `let x=1 in x+2` and `if c then s else s`. Add panic-mode recovery on missing `;` and on `if (c) { s }` (missing else): what synchronising tokens/states, and what partial AST to keep for later sema?

Chapter 4

Semantic Analysis and Type Systems

"Syntax asks 'can I write this?' Semantics asks 'what does it mean – and is it allowed?'." — Scopes, bindings, and types catch bugs before any code runs.

From zero: we build here, no type theory assumed. We define names, scopes, bindings, and types from sets and functions. Judgements, inference rules, and unification are introduced via pictures then formalism. No prior logic course is assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish lexical vs. dynamic scope and implement a scoped symbol table;

(L2) resolve names, detect shadowing, and report precise "undeclared identifier" diagnostics;

(L3) read and write typing judgements $\Gamma \vdash e : \tau$ and derivation trees;

(L4) implement Hindley–Milner inference $\mathcal{W}$ with unification and let-generalisation;

(L5) design coercions, overloading resolution, and gradual typing checks.

4.1 Names, Scopes, and Bindings

Intuition: a name is a use; a declaration is its binder. Scope is the region where that binder is visible. Like nested boxes—inner box shadows outer.

Definition 4.1 (Scope and binding)**.** A *declaration* binds identifier x to an entity (variable, function, type). A *use* is an occurrence of x. The *scope* of a declaration is the set of program points where uses resolve to it. *Lexical* (static) scope = nesting of blocks; *dynamic* scope = most recent call on stack (rare, error-prone). NTU languages use lexical scope.

Example 4.1 (Shadowing).

```
int x = 1;          // outer x
{ int x = 2;        // inner x shadows
  print(x);         // 2
}
print(x);           // 1
```

Two distinct entities share spelling `x`; resolution picks the innermost enclosing binding. A symbol table must model this.

Symbol table as a stack of maps. Enter scope → push new map; declare x → insert in top map (error if already there—redeclaration); lookup x → walk stack top-to-bottom; exit scope → pop. For efficient walking, chain entries via "next in scope" or hash with scope id. Fig. 4.1 shows tree vs. stack view.

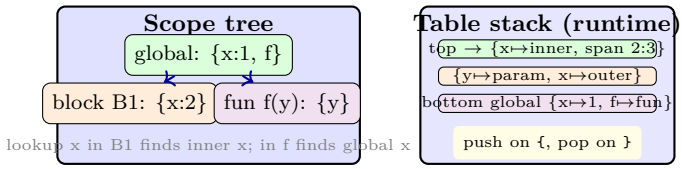


Figure 4.1: Scopes as nested boxes: tree view (declaration sites) and stack view (table push/pop while walking the AST). Shadowing is stack shadowing.

```
class SymTab:
    def __init__(self): self.stack = [{}]          # global scope
    def enter(self): self.stack.append({})
    def exit(self): self.stack.pop()
    def declare(self, name, info):
        if name in self.stack[-1]: raise Redeclare(name)
        self.stack[-1][name] = info
    def lookup(self, name):
        for scope in reversed(self.stack):
            if name in scope: return scope[name]
        raise Undeclared(name)
    # visitor: walk AST, enter/exit at block/let/fun nodes
```

4.2 Type Systems: Judgements and Rules

Intuition: types are lightweight specifications: "this expression will produce an `int`". A type system proves (at compile time) that no execution will confuse `int` with `string`.

Definition 4.2 (Types and judgements)**.** Base types: `int`, `bool`, `string`. Compound: $\tau_1 \to \tau_2$ (function), $\tau_1 \times \tau_2$ (pair), $[\tau]$ (list), $\forall\alpha.\tau$ (polymorphic). *Typing environment* Γ is a map $x_1 : \tau_1, \ldots, x_n : \tau_n$. Judgement $\Gamma \vdash e : \tau$ reads "under Γ, expression e has type τ".

Inference rules are fractions: premises above, conclusion below.

$$\text{T-Var}\ \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \qquad \text{T-Add}\ \frac{\Gamma \vdash e_1 : \texttt{int} \quad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 + e_2 : \texttt{int}}$$

$$\text{T-If}\ \frac{\Gamma \vdash c : \texttt{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \texttt{if}\ c\ \texttt{then}\ e_1\ \texttt{else}\ e_2 : \tau} \qquad \text{T-Abs}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.e : \tau_1 \to \tau_2}$$

A *derivation tree* composes rules; its root is the program's type. If no derivation exists, the program is ill-typed.

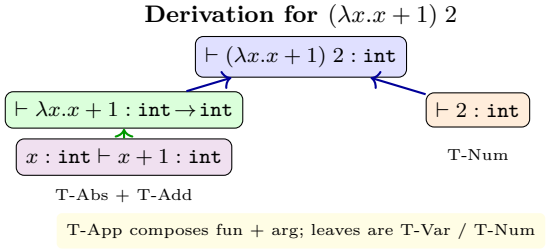


Figure 4.2: A typing derivation tree: each node is a judgement justified by an inference rule; composition mirrors the AST shape.

Example 4.2 (Ill-typed and why)**.** `if 3 then 1 else 0` fails: premise $\Gamma \vdash 3 : \texttt{bool}$ does not hold (`3` has `int`). Compiler error: "expected `bool`, got `int` in condition". Good errors quote the rule that failed and point at the offending subexpression's span.

4.3 Hindley–Milner Inference and Unification

Monorphic checks need annotations; polymorphic inference invents them. *Let-polymorphism*: `let id = \x.x in id 3` gives `id` type $\forall\alpha.\alpha \to \alpha$, instantiated differently per use.

Definition 4.3 (Unification)**.** Given equations $\tau_1 = \sigma_1, \ldots$, a *unifier* is a substitution S on type variables with $S(\tau_i) = S(\sigma_i)$. The *most general unifier* (mgu) is the least committing; Robinson's algorithm finds it by structural recursion, with *occurs check* $\alpha \notin \mathrm{FV}(\tau)$ to reject $\alpha = \alpha \to \alpha$.

Algorithm $\mathcal{W}(\Gamma, e)$ returns (S, τ) (substitution + type):

1. $e = x$: lookup $x : \forall\bar{\alpha}.\tau$ in Γ, freshen $\bar{\alpha}$ to new vars, return $(\emptyset, \tau')$.
2. $e = e_1\, e_2$: infer (S_1, τ_1) for e_1, (S_2, τ_2) for $S_1(e_2)$, fresh β, unify $S_2(\tau_1) = \tau_2 \to \beta$, return.
3. $e = \lambda x.e_1$: fresh β for x, infer $\Gamma, x : \beta \vdash e_1$, return $\beta \to \tau_1$.
4. $e = \texttt{let}\ x = e_1\ \texttt{in}\ e_2$: infer (S_1, τ_1) for e_1, generalise free vars not in $S_1(\Gamma)$ to $\forall$, infer e_2 under extended Γ, compose.

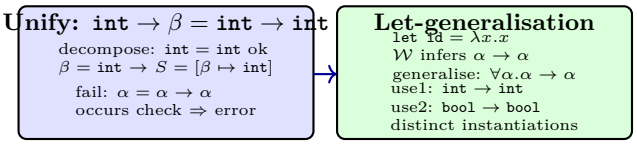


Figure 4.3: Left: unification $\beta = \texttt{int}$ succeeds, recursive occurs-check fails. Right: let-generalisation gives polymorphism; each use freshens $\forall$-vars.

```
def unify(t1, t2, subst):          # t = ('var',a) | ('con','int') | ('arr',l,r)
    t1, t2 = apply(subst,t1), apply(subst,t2)
    if t1 == t2: return subst
    if is_var(t1):
        if occurs(t1[1], t2): raise TypeError("occurs check")
        return compose({t1[1]: t2}, subst)
    if is_var(t2): return unify(t2, t1, subst)
    if t1[0]=='arr' and t2[0]=='arr':
        s = unify(t1[1], t2[1], subst)
        return unify(apply(s,t1[2]), apply(s,t2[2]), s)
    raise TypeError(f"cannot unify {t1} vs {t2}")
```

Coercions and overloading. Languages insert implicit coercions (`int` → `float`) via elaboration; overloading resolves + to integer-add vs. float-add vs. string-concat by type of operands (or type classes). Gradual typing adds `?` (unknown) with runtime checks at boundaries.

4.4 Chapter Notes

Symbol tables date to assemblers; scoped tables are due to Dijkstra (block structure). Hindley–Milner (1969/78) underlies ML, Haskell, and Rust's trait inference. Damas–Milner $\mathcal{W}$ is linear in practice; production compilers memoise and parallelise. Pierce's *Types and Programming Languages* is the rigorous reference; Appel Ch. 5 maps it to compiler implementation.

4.5 Exercises

E4.1 **Scope implementation.** Extend `SymTab` to support (a) function scopes where params are in function's scope not caller's, and (b) `let x= e1 in e2` where `x` not visible in `e1`. Give visitor pseudocode and a test with shadowing depth 3.

E4.2 **Derivation tree.** Derive $\vdash$ `let` $f = \lambda x.x$ `in` $(f\ 3,\ f$ `true`$)$: `int` $\times$ `bool` using T-Let, T-Abs, T-App. Mark where generalisation happens.

E4.3 **Unification trace.** Unify $(\alpha \to \alpha) \to \beta = (\texttt{int} \to \texttt{int}) \to \texttt{bool}$ stepwise; show substitution composition. Then show $\alpha \to \beta = \beta \to \alpha$ yields $[\beta \mapsto \alpha]$ or $[\alpha \mapsto \beta]$—why many mgus?

E4.4 **Occurs check.** Explain why $\mathcal{W}$ must reject $\lambda x.x\ x$ (self-application). Attempt to infer its type and pinpoint the equation $\alpha = \alpha \to \beta$ triggering occurs check.

E4.5 **Coercion elaboration.** Design insertion for `int` $\leq$ `float` with rule `int+float→float` (coerce left). Elaborate `(1 + 2.0) + 3` to explicit `toFloat` calls and give typing derivation; where would a coercion lattice place `int<:float`?

Chapter 5

Intermediate Representations: 3AC and SSA

"A good IR is half the compiler." — Between AST and assembly, a uniform, low-level yet machine-independent IR lets one optimiser serve many languages and targets.

From zero: we build here, no IR assumed. We define three-address code, basic blocks, control-flow graphs, dominance, and static single assignment from sets and graphs. Every transformation is shown on a running example: factorial.

Learning Outcomes

After this chapter you will be able to:

(L1) lower AST expressions/statements to three-address code with temporaries and labels;

(L2) partition 3AC into basic blocks and build the control-flow graph (CFG);

(L3) define dominance, immediate dominance, dominator tree, and dominance frontier;

(L4) convert a CFG to SSA form via ϕ-placement (Cytron) and renaming;

(L5) explain mem2reg, SSA destruction, and why SSA simplifies optimisation.

5.1 Three-Address Code

Intuition: an AST nests arbitrarily; 3AC flattens to "at most one operation per line" with explicit temporaries—like assembly without registers.

Definition 5.1 (3AC)**.** Instructions: $x \leftarrow y$ op z (binary), $x \leftarrow$ op y (unary), $x \leftarrow y$ (copy), `goto` L, `if` x relop y `goto` L, $x \leftarrow y[z]$ / $x[z] \leftarrow y$ (array), $x \leftarrow$ `call` $f(args)$, `return` x, `label` L. Each has ≤ 3 addresses (hence the name). Temporaries $t_1, t_2, \ldots$ hold intermediate results.

Example 5.1 (Lowering expression)**.** AST `a = b + c*2` → 3AC:

```
t1 = c * 2
t2 = b + t1
a  = t2
```

If source had `if (x < 10) y=1 else y=2`, we emit conditional branches and labels (next section). Short-circuit `a && b` lowers to branching, not bitwise `and`.

Lowering walks the AST, threading a fresh-temp counter and a label generator. For boolean contexts, we use *jumping code*: `E` in condition position emits jumps to true/false labels instead of a boolean value.

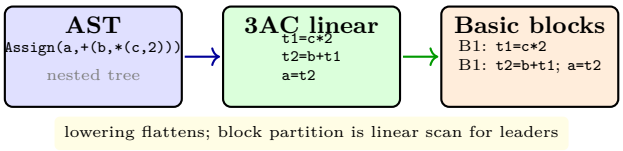


Figure 5.1: Lowering AST to linear 3AC then partitioning into basic blocks. Leaders are first instruction, jump targets, and instructions after jumps.

5.2 Basic Blocks and CFGs

A *basic block* is a maximal straight-line sequence with one entry (first instr) and one exit (last instr, a jump/branch/return). Partition by marking *leaders*: (i) first instruction, (ii) any label target, (iii) instruction after a jump. Blocks become CFG nodes; edges are "may jump to".

Example 5.2 (Factorial CFG)**.** Source `fact(n) { r=1; while(n>1){r=r*n; n=n-1} return r}` gives blocks: B1 `r=1`, B2 header `if n>1 goto B3 else B5`, B3 body `r=r*n; n=n-1; goto B2`, B5 `return r`. Edges B1→B2, B2→B3/B5, B3→B2. See Fig. 5.2.

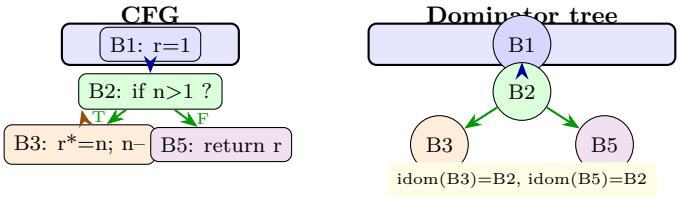


Figure 5.2: Factorial CFG (left) and its dominator tree (right). B1 dominates all; B2 dominates loop body and exit. Dominance frontier of B3 is {B2}—where ϕ for loop-carried `r,n` will appear.

5.3 Dominance and SSA

Intuition: in straight-line code, each variable has one value reaching each use. Branches and loops merge values. SSA makes the merge explicit with ϕ-functions: "this variable is whichever predecessor's value arrived".

Definition 5.2 (Dominance)**.** Node d *dominates* n (d **dom** n) if every path entry$\to n$ includes d. *Strict*, *immediate dominator* `idom` (closest strict dominator), and *dominator tree* (idom edges) follow. The *dominance frontier* $\mathrm{DF}(n) = \{y \mid n$ doms a pred of y but not y strictly$\}$—exactly where ϕ may be needed.

Dominator tree via Lengauer–Tarjan $O((V+E)\log(V+E))$; DF by walking.

Definition 5.3 (SSA)**.** Each variable assigned exactly once. Multiple assignments become versions $x_1, x_2, \ldots$; at control merges, $x_3 \leftarrow \phi(x_1, x_2)$ selects the incoming value. Fig. 5.3 shows factorial after SSA: `r1` before loop, `r2=`ϕ`(r1,r3)` at header, similarly `n2=`ϕ`(n1,n3)`.

Example 5.3 (Why ϕ lives at DF)**.** Consider diamond A: $x = 1 \to B, C \to D$: $y = x$. A dominates B, C but not D strictly (both B, C reach D). The merge at D is exactly the frontier of $\{B, C\}$ (where definitions from one side stop dominating the join). Inserting $\phi(x)$ at D makes y's use see the correct incoming version without searching predecessors.

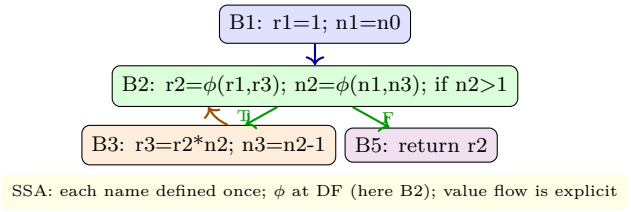


Figure 5.3: Factorial in SSA. Cytron placement inserts ϕ for `r,n` at DF of their definitions (B2); renaming then versions them. Optimisations now see defs dominate uses.

Cytron SSA construction. 1) Insert ϕ: for each variable x, let $Defs = \{blocks\ defining\ x\}$; iterated DF$^+$ (worklist adding DF of newly inserted) places ϕ for x at each frontier block. 2) Rename: DFS over dominator tree, maintaining version stacks per variable; on defining instruction assign fresh version and push; on use replace with stack top; on ϕ define new version; pop on exit. Complexity near-linear.

```
def ssa_rename(block, stacks, counter):
    # stacks[x] = list of current versions; counter[x] = next fresh id
    for phi in block.phis:                  # e.g. r2 = phi(r1,r3)
        v = new_version(phi.var, counter); stacks[phi.var].append(v); phi.dst=v
    for inst in block.insts:
        for i, use in enumerate(inst.uses): inst.uses[i]=stacks[use.var][-1]
        if inst.defs:
            v = new_version(inst.def.var, counter)
            stacks[inst.def.var].append(v); inst.def=v
    for succ in block.succs:                # fix phi operands
        for phi in succ.phis:
            if phi.var in stacks: phi.args[block.id]=stacks[phi.var][-1]
    for child in dom_tree[block.id]: ssa_rename(child, stacks, counter)
    for phi in block.phis: stacks[phi.var].pop()
    for inst in block.insts:
        if inst.defs: stacks[inst.def.orig].pop()
```

Why SSA helps. Def-use chains are explicit; dominance ⇒ cheap liveness; many optimisations become sparse. Destruction (out of SSA) replaces ϕ by copies on predecessor edges, coalescing to reduce moves.

From 3AC to SSA to target (mem2reg). Source variables may be *memory* (stack slots) initially: `alloca` + `load`/`store`. The *mem2reg* promotion identifies allocas whose address never escapes and whose loads/stores

are analysed as above; those become SSA registers, phi-inserted exactly as for `r,n`. This is often the first optimisation pass: after it, dataflow is sparse. LLVM's `-mem2reg` does this; GIMPLE's "out of SSA" must respect calling convention copies.

IR design choices. How high-level should IR be? LLVM IR keeps types and typed pointers for alias analysis; some IRs are untyped but add explicit bounds checks. Multi-level IR (MLIR) stacks dialects: high-level loops → affine → CFG → machine. The principle: keep IR that makes the next analysis easy, not that mirrors source syntax. That is why AST → 3AC → SSA is the canonical lowering pipeline.

Critical edges and splitting. An edge $u \to v$ is *critical* if u has ≥ 2 successors and v has ≥ 2 predecessors. ϕ-copy insertion or code motion on such edges needs a new block (edge splitting) to hold the inserted code without affecting other paths. Compilers split critical edges before SSA destruction and before many loop opts; the CFG shape after splitting has no critical edges, simplifying correctness proofs.

Example 5.4 (3AC with labels for factorial).

```
L1: r=1
L2: if n>1 goto L3 else L5
L3: r=r*n; n=n-1; goto L2
L5: return r
```

Leaders are `L1,L2,L3,L5`; blocks are maximal sequences between leaders. The CFG edges are exactly the `goto/if-goto` targets plus fall-through. This labelled view is what the partition pass emits before building CFG successor lists.

5.4 Chapter Notes

3AC is due to UNCOL dreams; LLVM IR and GCC GIMPLE are modern instances. SSA was invented by Rosen–Wegner–Zadeck (1988) and normalised by Cytron et al. (1991). Dominance via Lengauer–Tarjan (1979) remains the fastest. For practice, "SSA book" (Boissinot et al.) and Appel/Lattner give construction details; mem2reg (promoting stack slots to SSA) is often a student's first pass.

5.5 Exercises

E5.1 **Lower + partition.** Lower `while(x<10){ if(y>0) x=x+1 else x=x+2}` to 3AC with jumping code, then partition into basic blocks and draw CFG. Identify leaders and critical edges.

E5.2 **Dominance by dataflow.** Write iterative equations $\mathrm{Dom}(entry) = \{entry\}$ and $\mathrm{Dom}(n) = \{n\} \cup \bigcap_{p \in pred(n)} \mathrm{Dom}(p)$; iterate on factorial CFG to fixed point. Prove it equals path-definition of dominance.

E5.3 **DF and ϕ placement.** For CFG diamond $A \to B, C$, $B, C \to D$, with defs of `v` in B and C only, compute $\mathrm{DF}(B)$, $\mathrm{DF}(C)$, and iterated DF; where does ϕ(v) appear? What if def also in A?

E5.4 **Rename walkthrough.** Rename factorial CFG of Fig. 5.3 starting stacks {r:[r0], n:[n0]}. Show stack heights at each block and that no use sees an undefined version.

E5.5 **SSA destruction.** After optimisation, factorial SSA has `r3=r2*n2` in B3. Replace ϕ in B2 by copies on edges B1→B2 (`r2=r1`) and B3→B2 (`r2=r3`); draw resulting non-SSA CFG and argue semantics preserved (plus where coalescing removes a copy).

Chapter 6

Optimisation: Dataflow, Inlining, and Loops

"Optimisation is semantics-preserving rewriting guided by analysis." — Analyse facts on the CFG, then transform where profitable and safe.

From zero: we build here, no dataflow assumed. We define lattices, transfer functions, meet, and fixed-point iteration from sets and orderings. Reaching definitions, liveness, DCE, CSE, inlining, and loop opts are motivated by before/after pictures.

Learning Outcomes

After this chapter you will be able to:

(L1) formulate a dataflow analysis as a lattice + flow equations and solve to fixed point;

(L2) compute reaching definitions and liveness by hand on a CFG and use them for DCE/CSE;

(L3) apply constant propagation/folding and prove it preserves semantics;

(L4) decide when to inline with a cost heuristic and explain code-bloat vs. call overhead;

(L5) optimise loops via LICM, unrolling, and strength reduction with legality checks.

6.1 Dataflow Framework

Intuition: each block computes a fact ("which definitions reach here?", "which variables are live?"). Facts flow along edges until they stabilise.

Definition 6.1 (Lattice framework)**.** A lattice $(L, \sqsubseteq, \sqcup, \sqcap, \top, \bot)$ is a partial order where joins $\sqcup$ (least upper bound) and meets $\sqcap$ exist. For may-analyses we use $\sqcup$ (union); for must-analyses $\sqcap$ (intersection). A *transfer* $f_B : L \to L$ for block B has form $f_B(X) = \mathrm{Gen}_B \cup (X \setminus \mathrm{Kill}_B)$ (bitvector). Flow equations: $\mathrm{In}[B] = \bigsqcup_{P \in pred(B)} \mathrm{Out}[P]$

(forward) or $\text{In}[B] = f_B(\text{Out}[B])$ with $\text{Out}[B] = \bigsqcup_{S \in succ(B)} \text{In}[S]$ (backward). Solve by worklist iteration from $\bot$ until fixed point (monotone f, finite lattice $\Rightarrow$ converges).

Example 6.1 (Reaching definitions)**.** Fact per point: set of defs $d : x \leftarrow \ldots$ that may reach without intervening kill. Gen_B = defs in B that are not killed later in B; Kill_B = all other defs of same variable(s). Forward, $\sqcup = \cup$, $\text{In}[entry] = \emptyset$. Use to detect unused defs (no use reached) or to feed constant prop.

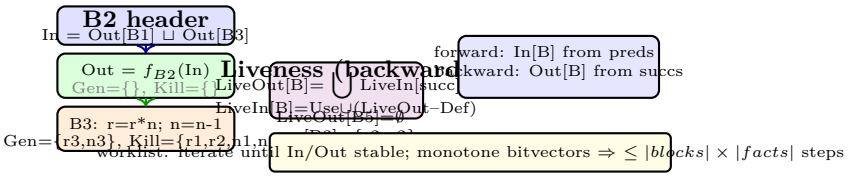


Figure 6.1: Flow equations on factorial CFG: forward reaching defs and backward liveness. The worklist iterates Gen/Kill transfer until a fixed point; precision is maximal for distributive frameworks.

6.2 Classical Optimisations

Liveness + dead-code elimination (DCE). Variable x is *live* at point p if some path $p \to use(x)$ has no intervening def. Compute backward as in Fig. 6.1. If instruction $x \leftarrow \ldots$ defines x but $x \notin$ LiveOut, it is dead—delete it (if no side effects). Iterate; one deletion may kill another.

Available expressions & CSE. Expression e is *available* at p if every path entry$\to p$ computes e and no kill intervenes (must-analysis, $\sqcap = \cap$, forward). If $a + b$ available, reuse prior temp instead of recomputing (common subexpression elimination). SSA makes the check syntactic: same value numbers.

Constant propagation & folding. Lattice per variable: $\top$ (unknown) $> c$ (constant) $> \bot$ (conflicted). Transfer: $x \leftarrow c$ sets c, $x \leftarrow y$ op z maps via constant table. Fold $2 * 3 \to 6$ after propagation; conditional branches become unconditional when guard is constant (enabling block removal).

Example 6.2 (Liveness on factorial)**.** Compute: LiveOut[B5]=∅, LiveOut[B3]={r2,n2}?? Actually return uses r2, so LiveIn[B5]={r2}, propagate backward through branch to B2, etc. Fig. 6.2 shows lattice climb. If we insert `t = n*0` in B3 and `t` never used, liveness marks it dead and DCE removes it.

Example 6.3 (CSE via value numbering)**.** Before: `t1=b+c; t2=b+c; d=t1*t2`. Value number: `b+c` hashes to v_7; second compute hits v_7 so `t2=t1` (CSE). After copy prop and DCE, one add remains. SSA makes this linear: names are versions, so syntactic equality implies value equality (modulo commutativity).

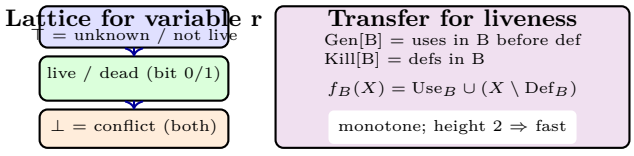


Figure 6.2: Bitvector lattice for liveness (powerset of variables) and transfer f_B. The join is $\cup$ (may-live); $\bot = \emptyset$, $\top$ =all variables; iteration climbs until fixed point.

```python
def liveness(cfg):
    live_in = {b:set() for b in cfg}
    live_out = {b:set() for b in cfg}
    worklist = list(reversed(cfg.postorder()))
    while worklist:
        b = worklist.pop()
        out = set().union(*(live_in[s] for s in b.succs)) if b.succs else set()
        inn = b.use | (out - b.defs)
        if inn != live_in[b] or out != live_out[b]:
            live_in[b], live_out[b] = inn, out
            worklist.extend(b.preds)
    return live_in, live_out
# DCE: if 'x = ...' and x not in live_out[block after inst] -> remove
```

6.3 Inlining and Loop Optimisation

Inlining. Replace `y = f(x)` by body of `f` with params substituted and returns wired to `y`. *Why:* removes call overhead, exposes context to intraprocedural opts (CSE, constant prop), enables devirtualisation. *When:* heuristic: small callee (<50 IR instrs), hot call site (profile), single call site, not recursive. Cost: code bloat, I-cache pressure, compile time. Partial inlining and outlining trade finer.

Loop-invariant code motion (LICM). If instruction in loop computes same value each iteration (operands invariant or defined outside loop), hoist it to preheader. Condition: dominates all loop exits where the value is used, and loop header dominates the instruction (so not speculative). Fig. 6.3 shows LICM + unrolling.

Unrolling and strength reduction. Unrolling repeats body k times, reducing branch and enabling vectorisation, at cost of size. Strength reduction replaces $i * 8$ by $i << 3$ or induction variable $t \leftarrow t + K$ stepping with loop.

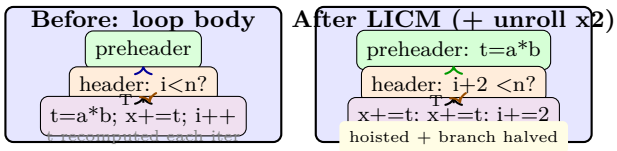


Figure 6.3: LICM hoists loop-invariant `a*b` to preheader; unrolling by 2 halves branches and exposes instruction-level parallelism. Both need dominance and legality checks.

```python
def licm(loop, dom):
    preheader = loop.preheader
    for block in loop.blocks:
        for inst in list(block.insts):
            if not is_invariant(inst, loop): continue
            if not dominates(inst.block, loop.exits): continue
            # hoist copy to preheader end (before jump to header)
            preheader.insts.insert(-1, inst.copy())
            block.insts.remove(inst)
# loop unrolling (k=2) duplicates body, rewires backedge, adds remainder
```

Phase ordering caveat. No single order dominates: CSE enables LICM (by equalising names), LICM enables DCE (by moving the last use), DCE enables inlining (by shrinking callee). Real pipelines iterate: run CFG opts to fixed point, then re-run loop opts after inlining. Budget caps prevent infinite loops: "optimise until no change or 3 rounds" is typical. Correctness requires each pass preserves the commuting square (Ch. 1); buggy LICM that hoists a trapping divide changes semantics.

Profile-guided optimisation (PGO). Instrumented builds count edge frequencies or sample via perf; the compiler then inlines hot calls, unrolls hot loops, lays out blocks for fall-through (hot path straight), and splits cold code to distant pages. PGO can beat static heuristics by 10–15% on server workloads, explaining why Ch. 8's JIT tiers are "always PGO".

6.4 Chapter Notes

Dataflow was formalised by Kildall (1973) and Kam–Ullman; SSA sparse analyses are due to Wegman–Zadeck. Inlining heuristics trace to Davidson–Hollander; modern PGO (profile-guided) inlines hot paths first. Loop opts draw from Allen–Cocke (1971) and Wolf–Lam. The optimisation pipeline order matters—e.g., inline → CSE → LICM → DCE—so compilers iterate "optimise until fixed point or budget".

6.5 Exercises

E6.1 **Reaching defs fixed-point.** For diamond CFG (B1→B2,B3→B4), defs: B1:`x=1`, B2:`x=2`, B3:`y=3`, B4:`z=x+y`. Compute Gen/Kill, iterate In/Out to fixed point. Which defs reach B4's use of `x`?

E6.2 **Liveness + DCE.** CFG B1:`a=1; b=2`, B2:`c=a+b; d=5`, B3:`return c`. Compute liveness; prove `d=5` dead and safe to delete, but `b=2` not. What if B3 returned `c+d`?

E6.3 **Constant prop lattice.** Draw lattice for value `v` with elements $\{\top, 0, 1, 2, \ldots, \bot\}$. Show transfer for `x = y + 1` when `y` is 3 vs. $\bot$ vs. $\top$; why does $\bot$ poison?

E6.4 **Inlining cost model.** Callee has 40 instrs, 1 call site, hot (10k calls/s). Inlining saves $\tilde{1}5$ cycles/call but grows I-cache by 40 instrs (∼160B). If I-cache miss costs 200 cycles and miss rate rises 0.1%, break-even? Propose a threshold.

E6.5 **LICM legality.** Loop `for(i=0;i<n;i++) { t=a/b; a[i]=t }` with `b` loop-invariant but `a` modified in loop and may alias `b`'s storage. Is `a/b` invariant? What alias analysis or speculation (with guard) is needed to hoist safely?

Chapter 7

Register Allocation and Code Generation

"Infinite virtual registers, k physical ones: colour the interference graph or spill." — Then tile trees to select instructions and honour the calling convention.

From zero: we build here, no backend assumed. We define target instructions, calling convention, liveness, interference, graph colouring, spilling, linear scan, and instruction selection from sets/graphs. Pictures precede algorithms.

Learning Outcomes

After this chapter you will be able to:

(L1) compute liveness intervals and build the interference graph;

(L2) colour the graph via Chaitin–Briggs (simplify, spill, select) and coalesce moves;

(L3) spill with reloads, estimate spill cost, and iterate;

(L4) contrast graph colouring vs. linear scan and pick per target;

(L5) tile IR trees for instruction selection (maximal munch / DP) and emit correct prologue/epilogue.

7.1 Why Registers Are Scarce

Intuition: IR uses unbounded temporaries $t_1, t_2, \ldots$; a real CPU has k general-purpose registers (e.g., $k = 16$ on x86-64, 32 on ARM, but fewer caller-saved). Two temporaries *interfere* if both live at some point—they cannot share a register. The problem is graph colouring.

Definition 7.1 (Liveness for intervals)**.** For CFG in SSA or after SSA destruction, t is *live* at point p if a path $p \to use(t)$ has no intervening def. Its *live interval* $[def(t), last\_use(t)]$ over linear order (after block linearisation) conservatively covers where it interferes. Exact interference is per-program-point, not just interval (holes where t not live inside interval are okay to reuse if using graph, not linear scan).

Example 7.1 (Intervals for `a=b+c*2`)**.** 3AC: $t_1 = c * 2$, $t_2 = b + t_1$, $a = t_2$. Linear order: 1:`t1`, 2:`t2`, 3:`a`. Live: c live at 1, b live at 2, t_1 live $1 \to 2$, t_2 live $2 \to 3$. So b interferes with t_1 (both live at 2? b used at 2, t_1 live through 2), but c not with t_2.

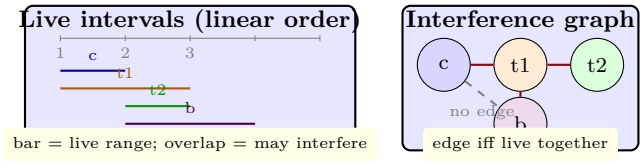


Figure 7.1: Live intervals over linearised blocks (left) induce the interference graph (right). Graph colouring is precise; linear scan uses only interval overlap and may over-approximate.

7.2 Graph Colouring Allocation (Chaitin–Briggs)

Intuition: k-colour the graph where k = register count. If degree $< k$, a node can always be coloured after neighbours (remove it, colour rest, then pick a free colour). If all degrees $\geq k$, pick a spill candidate, spill it to stack, and retry.

Briggs optimistic variant. Simplify: repeatedly remove any node with degree $< k$ onto stack (if none, pick spill candidate by cost heuristic $uses + defs$ weighted by loop depth 10^{depth}, push it optimistically anyway). Then select: pop stack, assign first colour not used by already-coloured neighbours; if none, this node is the actual spill—insert reload before each use and store after each def, rewrite, and repeat.

Coalescing. Move $x \leftarrow y$ where x, y do not interfere can be coalesced (same register, move deleted) if the merged node stays $< k$-colourable (Briggs: combined neighbours $< k$ high-degree neighbours). Iterative coalescing interleaves simplify/coalesce for best elimination.

Example 7.2 (4-colour walkthrough with spill)**.** Graph: {t1–c, t1–b, t1–t2, a–t2} plus many neighbours. With $k = 2$ registers, simplify removes c, a (degree 1), leaving core $\{t1, b, t2\}$ where each degree 2 ($= k$)—no $< k$ node. Pick spill least cost, say b (cold). Stack $[c, a, b, t1, t2]$; selecting colours: $t2 \to R0$, $t1$ interferes $t2$ so $R1$, b interferes $t1$ so $R0$ (optimistically coloured—no spill needed!). If all 3 had pairwise edges (triangle) with $k = 2$, the last popped *would* spill and be reloaded from stack slot.

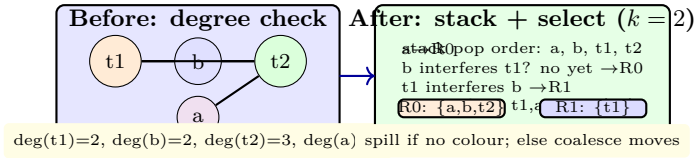


Figure 7.2: Briggs simplify/select with $k = 2$: remove low-degree nodes, then assign colours on the way back. Spill only if truly needed; coalescing would merge a move-related pair if safe.

```
def colour(interf, k, move_pairs):
    stack, removed = [], set()
    # simplify + coalesce loop
    while len(stack)+len(removed) < len(interf):
        v = pick_low_degree(interf, removed, k) or pick_spill(interf, removed)
```

```
        stack.append(v); removed.add(v)
        coalesce_around(v, interf, move_pairs, k)
    colour_of = {}
    spilled = set()
    while stack:
        v = stack.pop()
        used = {colour_of[w] for w in interf[v] if w in colour_of}
        free = [c for c in range(k) if c not in used]
        if free: colour_of[v]=free[0]
        else: spilled.add(v)          # insert spill code and retry
    return colour_of, spilled
```

7.3 Linear Scan and Instruction Selection

Linear scan (Poletto–Sarkar). Sort intervals by start; sweep maintaining *active* intervals (those overlapping current start). If $|\text{active}| = k$, spill the interval with farthest end (or current if it ends farthest—then current spills, not active). Fast $O(n \log n)$, used by JITs (HotSpot, V8) where compile speed beats last-percent quality. Holes are ignored (pessimistic).

Instruction selection by tiling. IR tree (e.g., $+(b, *(c, 2))$) is tiled with target patterns: $c * 2 \to \texttt{sll}\ c, 1$ (strength reduce), $b + t_1 \to \texttt{add}$. Maximal-munch picks largest tile greedily; optimal DP picks min-cost cover (BURS). Fig. 7.3 shows tiling.

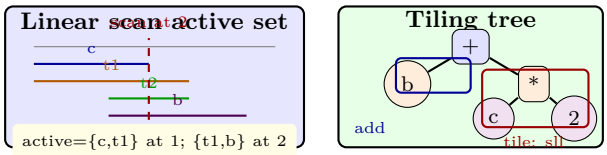


Figure 7.3: Left: linear scan sweep with active set; spill farthest-ending. Right: IR tree $b + (c * 2)$ tiled as `sll` for $c * 2$ plus `add` for $b + t_1$. Cost DP chooses minimal tiles.

Calling convention & prologue/epilogue. Caller-saved vs. callee-saved registers, argument registers ($a0$–$a3$ on MIPS, $r0$–$r3$ on ARM), return $v0$, stack alignment (16B), and spill slots all emitted in function prologue (`push ra, fp; sub sp, framesize`) and epilogue. Correctness requires spilled temps reload before every use.

```
// MIPS-like emitter after allocation: R0 -> $t0, R1 -> $t1
// 3AC: t1 = c * 2; t2 = b + t1; a = t2  with t1->R1, b->R1? conflict so b->R0
lw $t0, b           // b in R0
lw $t1, c           // c in R1
sll $t1, $t1, 1     // t1 = c*2  (tiling picked sll)
add $t0, $t0, $t1   // t2 reuses $t0
sw $t0, a
```

7.4 Chapter Notes

Chaitin (1982) cast allocation as colouring; Briggs (1992) made it optimistic with coalescing; George–Appel refined iterated coalescing. Linear scan (Poletto–Sarkar 1999) trades a few percent runtime for 10× compile speed—why JITs love it. Tiling theory is Aho–Johnson (1976) and Fraser–Hanson (BURG). For ARM/x86 specifics, consult ABI docs and Appel Ch. 10–11.

7.5 Exercises

E7.1 **Interf from liveness.** CFG with blocks B1:`x=1`, B2:`y=x+2`, B3:`z=x+y`, edges B1→B2,B3, B2→B3. Compute LiveIn/Out, then interference edges. Which pairs interfere? Why does interval approximation add a false edge?

E7.2 **Briggs on triangle.** Graph K_3 (3 nodes clique) with $k = 2$. Simulate simplify: degree < 2? none, pick spill cheapest, push, stack, then select and show one node must spill. What spill code inserted? Second iteration?

E7.3 **Coalescing safety.** Move `a=b` with interference edges a–c, b–c only, $k = 2$. Is coalescing a,b safe by Briggs criterion (combined high-degree neighbours $< k$)? Show merged graph and colour it with 2 colours, move gone.

E7.4 **Linear scan spill choice.** Intervals: a$[1, 4)$, b$[2, 3)$, c$[2, 6)$, d$[5, 7)$ with $k = 2$. Sweep sorted by start: at 2 active {a} size 1 add b →2, add c →3$> k$ so spill farthest end (c ends 6 vs a 4 vs b 3: spill c). Trace full sweep and final mapping.

E7.5 **Tiling DP.** IR tree $(b + c) * (d + 2)$ with patterns: add→`add` cost 1, mul→`mul` cost 2, mul by 2→`sll` cost 1. Compute minimal tile cost via DP (postorder: leaf cost 0, plus pattern). Which mul becomes sll?

Chapter 8

Advanced Topics: JIT, GC, and Incremental Compilation

"Compile late, collect dead, rebuild little." — Just-in-time tiers, automatic memory management, and incremental builds close the loop from theory to production toolchains.

From zero: we build here, no runtime assumed. We define interpreters, JIT tiers, deoptimisation, GC roots/reachability, write barriers, and build DAGs from first principles. Every mechanism is shown as a small state machine before formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) contrast AOT, interpreter, baseline JIT, and optimising JIT with tier transitions;

(L2) explain inline caches (IC), on-stack replacement (OSR), and deoptimisation guards;

(L3) define GC roots, reachability, tri-colour invariant, and compare mark–sweep/copying/generational;

(L4) implement a write barrier and explain why it preserves the invariant for concurrent/incremental GC;

(L5) design an incremental compiler: file DAG, fingerprinting, and LSP-style edit → re-analyse loop.

8.1 Just-in-Time Compilation

Intuition: AOT compiles once before run; interpreter runs immediately but slowly; JIT starts as interpreter, profiles hot spots, then compiles hot code with speculative opts—and falls back if speculation wrong.

Tiers. Modern VMs (HotSpot, V8, JSC) have: Tier 0 interpreter (fast start, collects counts); Tier 1 baseline JIT (fast compile, no opts, inline caches); Tier 2 optimising JIT (slow compile, inlines, LICM, vectorises, with

guards). Fig. 8.1 shows the pyramid.

Speculation and guards. Optimiser assumes "this call target never changes" or "this variable is always int" based on profile. It emits a *guard* (cheap type/predicate check); if guard fails, *deoptimise*: map optimised frame back to interpreter/baseline state (register → stack slots, OSR metadata) and resume without the bad assumption. Correctness: guarded fast path plus slow bailout equals unoptimised semantics.

Inline caches (IC). At call site `obj.m()`, first execution does full lookup; IC caches class → method, emitting `if class==cached then direct call else lookup`. Monomorphic IC (one class) becomes polymorphic (few) then megamorphic (hashtable). ICs are patched in place by JIT.

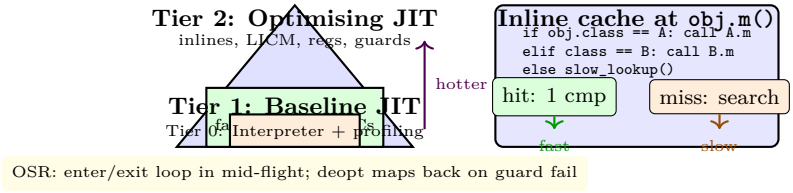


Figure 8.1: JIT tiers (interpreter → baseline → optimising, with OSR/deopt) and inline cache fast path vs. slow lookup.

```
# Baseline JIT stub with IC patch point (pseudo-x86)
def emit_call_site(obj_reg, method_name, ic_slot):
    emit(f"cmp [obj_reg+class_off], {ic_slot.cached_class}")
    emit(f"je  {ic_slot.cached_target}")        # fast path
    emit(f"call slow_lookup  # updates IC")     # miss: lookup + patch
    emit(f"{ic_slot.cached_target}:")
    emit("call rax")                            # indirect via cached addr
# Optimising JIT adds guard:  cmp type; jne deopt_label
```

8.2 Garbage Collection

Intuition: manual `malloc/free` leaks or double-frees; GC automates reclamation by reachability from roots (stack, globals, registers). Unreachable = garbage, regardless of `free` calls.

Mark–sweep. Two phases under stop-the-world pause: *mark* traverses object graph from roots (DFS/BFS) marking reachable; *sweep* scans heap linearly, linking unmarked blocks into free list. No moving—fragmentation handled by free-list (first/best fit) or mark–compact (slide survivors). Cost O(heap) per cycle, pause proportional to live set + heap size.

Copying / semi-space. Heap split in half: from-space (allocation) and to-space (empty). Minor GC copies live objects to to-space via Cheney scan (breadth-first with forwarding pointers), then flips spaces. Allocation is bump pointer (fast); fragmentation zero; cost O(live), not heap. Trade: half heap wasted.

Generational hypothesis. "Most objects die young." Young generation (Eden) collected frequently by copying (cheap, most die); survivors promoted to old generation, collected rarely by mark–sweep/compact. Write barriers (see below) remember old→young pointers so minor GC knows roots without scanning old heap.

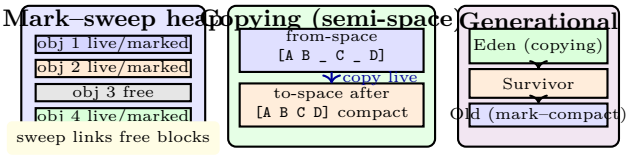


Figure 8.2: GC strategies: mark–sweep marks reachable then sweeps garbage; copying compacts live to to-space; generational applies copying to young (frequent) and mark–compact to old (rare).

Tri-colour invariant. For incremental/concurrent GC, objects are white (unreached), gray (reached but children not scanned), black (reached and scanned). Invariant: no black→white edge. Mutator (program) must maintain it via *write barrier*: on `p.f = q`, if `p` black and `q` white, either shade `q` gray (Dijkstra barrier) or shade `p` gray (Steele) or log the slot. Without barrier, concurrent marking could miss a newly installed white object.

Safepoints and root enumeration. Collector must find roots: stack frames contain pointers at known offsets (described by stack maps emitted by codegen, Ch. 7). At a *safepoint* (call, allocation, loop backedge) the mutator pauses, roots are enumerated, and marking proceeds. JIT must emit maps per safepoint; otherwise a register holding a pointer would be invisible and the object wrongly reclaimed. That is why GC-aware codegen (Ch. 7) and GC (this chapter) co-design.

```
// Dijkstra write barrier (incremental update)
void write_barrier(Object *p, Field f, Object *q) {
    p->field[f] = q;
    if (is_black(p) && is_white(q)) {
        shade_gray(q);          // q reachable now, must be scanned
        // or push to mark stack
    }
}
// Steele barrier alternative: shade_gray(p)  // rescan p
```

8.3 Incremental and Language-Server Compilation

Recompiling the world on each keystroke is wasteful. Incremental compilers track a *build DAG* over files/packages; each node has a fingerprint (hash of content + dependencies' fingerprints). On edit, only dirty nodes and transitive dependents re-run.

File DAG + fingerprints. For SCCs (circular imports), analyse together. Fingerprints use content hash plus sorted dependency fingerprints (like Bazel/Mercury). If fingerprint unchanged, downstream need not recompile even if dependency recompiled (stable interface). Caching of parse/typed IR per file (as in Rust `salsa`, OCaml `dune`) makes re-analyse O(changed) not O(world).

LSP loop. Editor sends `didChange` (incremental text edits); server re-lexes only affected lines (states at line boundaries cached), re-parses with error recovery keeping old tree's unchanged suffix, re-resolves names/types incrementally via dependency tracking, and pushes `diagnostics` + `completions`. Hot reload (for JIT, Ch. 8) patches running code via OSR without restart. The key data structure is the *incremental computation graph* (salsa): each query (parse file, type-check) is memoised by fingerprint; on edit only dirty queries recompute, others return memoised values. Red–green marking propagates.

Build vs. language incrementality. Build DAG incrementality is file-grained; language incrementality is finer (per function/type). Production Rust Analyzer tracks per-item fingerprints so editing one function does not re-type-check the whole crate—only dependents via type dependency graph (not just file graph). The same principle scales to whole-program optimisation via thinLTO: summaries give per-module fingerprints for cross-module inlining.

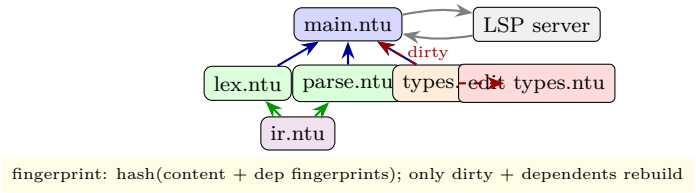


Figure 8.3: Incremental build DAG: edit to `types.ntu` dirties it and `main.ntu` but not `lex/parse/ir` if their fingerprints stable. LSP keeps the DAG live in memory for sub-100 ms diagnostics and completion.

8.4 Chapter Notes

JIT history: Self (1987) pioneered ICs and tiered compilation; HotSpot (1999) productised it; V8/TurboFan and JSC/F TL refined tiering and OSR. GC: McCarthy (1960) mark–sweep, Cheney (1970) copying, Ungar (1984) generational, Dijkstra (1978) tri-colour + barriers; Jones–Hosking–Moss (*GC Handbook*) is the reference. Incremental compilation draws from build systems (Make, Bazel) and language servers (Eclipse JDT, Rust Analyzer's salsa). The future is MLIR-style multi-level incremental JITs with GC-aware allocation.

8.5 Exercises

E8.1 **Tier thresholds.** Interpreter 10 M ops/s, baseline JIT 100 M ops/s but costs 5 ms to compile, optimising JIT 500 M ops/s but costs 50 ms. Function runs 100 ms interpreted. Which tier wins for 10 calls vs. 10 000 calls? Model total time = compile + run/prof.

E8.2 **IC patching.** Site `p.foo()` sees classes A×800, B×150, C×50. Design monomorphic → polymorphic → megamorphic transitions; at what miss rate does hash dispatch beat linear IC chain? Sketch patched assembly.

E8.3 **GC accounting.** Heap 100 MB, live 20 MB, pointer mutation rate low. Compare mark–sweep pause ($\propto$ heap) vs. copying pause ($\propto$ live) vs. generational (Eden 10 MB, 90% dies). Estimate pause times if marking 100 MB takes 10 ms and copying 20 MB takes 4 ms.

E8.4 **Write barrier correctness.** Show a race where mutator does `black_obj.f = white_obj` after collector scanned `black_obj` but before marking `white_obj`, losing reachability without barrier. Show how Dijkstra barrier (shade white) fixes it; why Steele (shade black) also works but rescans more?

E8.5 **Incremental fingerprint.** Three files: `ir` imports nothing, `parse` imports `ir`, `main` imports `parse,ir`. Hashes: ir=v1, parse=v2+hash(ir), main=v3+hash(parse,ir). Edit ir from v1→v1' with same interface hash (e.g., comment). Which files recompile? Which if interface hash changes?